DS	**Session**	Hashing	**Question Information**	Level 1 Challenge 97

Problem Description

Given an array A of N integers. Now, you have to output the sum of unique values of the maximum subarray sum of all the possible subarrays of the given array A.
Note: Subarray means contiguous elements with at-least one element in it.

Constraints

$1 \leq N \leq 2000$
$0 \leq |A_i| \leq 10^9$

Input Format

The first line of the input contains a single integer N, the total number of elements in array A.
The next line of the input contains N space-separated integers representing the elements of the array.

Output Format

The only single line of the output should contain a single integral value representing the answer to the problem.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4
5 -2 7 -3
```

EXPECTED OUTPUT

```
17
```

Test Case 2

INPUT (STDIN)

```
15
2 3 4 5 6 7 3 4 2 7 6 8 6 4 9
```

EXPECTED OUTPUT

```
2021
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int NA[N];
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

555

Test Case 3

NLOC

73

Code

C unique_subarray_max.c

```c
#include <stdio.h>
#include <stdlib.h>

long long max_sub_sum(int arr[], int l, int r) {
    long long max_so_far = arr[l];
    long long curr = arr[l];
    for (int i = l + 1; i <= r; i++) {
        curr = (curr + arr[i] > arr[i]) ? curr + arr[i] : arr[i];
        if (curr > max_so_far) max_so_far = curr;
    }
    return max_so_far;
}

int cmp_ll(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int NA[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &NA[i]);
    }

    int total_sub = N * (N + 1) / 2;
    long long* max_sums = (long long*)malloc(total_sub * sizeof(long long));
    int ptr = 0;

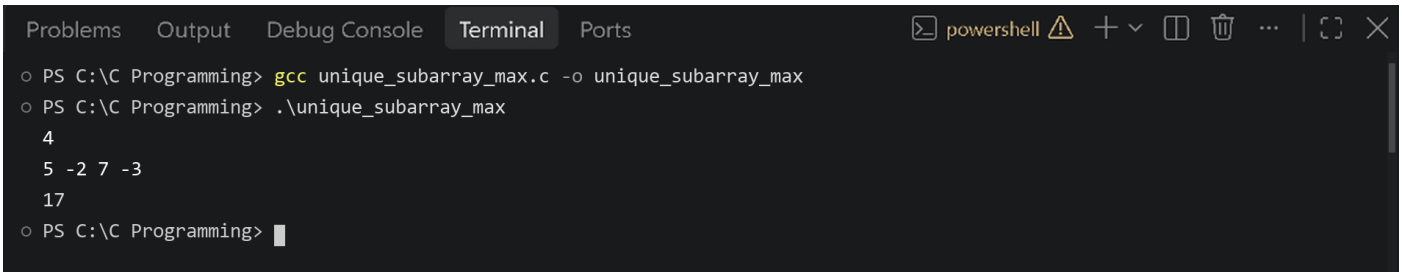
    for (int i = 0; i < N; i++) {
        for (int j = i; j < N; j++) {
            max_sums[ptr++] = max_sub_sum(NA, i, j);
        }
    }

    qsort(max_sums, ptr, sizeof(long long), cmp_ll);

    long long unique_sum = 0;
    for (int i = 0; i < ptr; i++) {
        if (i == 0 || max_sums[i] != max_sums[i - 1]) {
            unique_sum += max_sums[i];
        }
    }

    printf("%lld\n", unique_sum);
    free(max_sums);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc unique_subarray_max.c -o unique_subarray_max
PS C:\C Programming> .\unique_subarray_max
4
5 -2 7 -3
17
PS C:\C Programming>